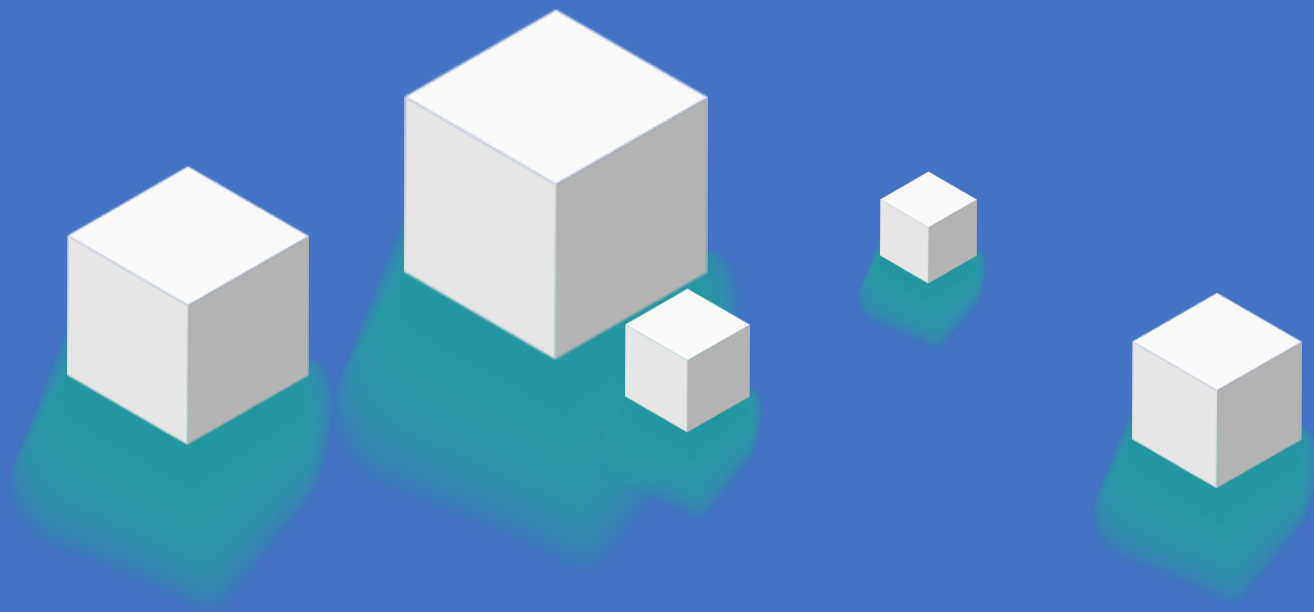


企业级AI部署

从硬件选型到框架选择



>> 今天的学习目标

从硬件选型到框架选择

- 企业级AI硬件选型

CPU vs. GPU

LLM的矩阵运算

LLM推理中的数据搬运

选型建议

资源配比

- 主流部署框架

从请求级调度 到 Token级调度

CASE: Ollama本地部署大模型

CASE: vllm高效部署

SGLang框架

- 结构化信息

Outlines工具

结构化输出 (OpenAI兼容)

企业级AI硬件选型与规划

CPU vs. GPU

架构差异的本质

- CPU:

设计初衷是处理顺序指令，擅长复杂逻辑控制和串行计算，核心数较少（数十核），但单核能力强

- GPU:

由数千个小而高效的核心组成 => 适合处理LLM的并行矩阵运算。

以NVIDIA H100为例，拥有16896个FP32 CUDA核心

RTX 4090 拥有 16384个 CUDA核心，中国特供版4090D为14592个。

CUDA 是NVIDIA推出的并行计算平台和编程模型，全称为Compute Unified Device Architecture。

CUDA核心是GPU的基本计算单元，专为并行计算设计，能同时执行大量简单运算

LLM的矩阵运算

Thinking: LLM 为什么需要矩阵运算?

LLM的核心是将文本转化为高维向量进行处理。

每个词会转为一个向量（如4096维），整句话就形成一个矩阵。

=> Transformer架构通过矩阵运算来捕捉词与词之间的复杂关系。

比如：输入句子有100个token，每个转为1×4096向量，堆叠成100×4096的矩阵。

注意力机制计算 token A与token B的相关度，本质是Query矩阵 × Key矩阵的乘法。

当模型有1750亿参数 => 处理的矩阵运算，计算量极其庞大。

LLM的矩阵运算

Thinking: 如果句子有100个token, 是直接生成一个 4096向量, 还是生成 100×4096 的矩阵?

每个token都有独立的向量。

=> LLM需要理解每个词的位置和含义, 以及它们之间的关系。如果整句压缩成一个向量, 就丢失了哪个词在前、哪个词在后的顺序信息。

具体过程:

- 句子有100个token $\rightarrow$ 生成100个4096维向量 (每个token一个)
- 这100个向量堆叠成 100×4096 矩阵输入Transformer

LLM的矩阵运算

Thinking: 句子会变成单个向量吗?

在LLM中不会。模型输出的也是100个向量（每个位置对应一个token的表示）。

但如果你想把整句话压缩成一个句向量，需要额外做池化操作

=> 比如对所有token向量取平均 => 得到一个4096维的句向量，用于计算相似度，或者分类等下游任务。

Embedding模型

Thinking: Embedding模型是什么？

Embedding模型（如BGE、M3E）本质上就是LLM + 池化层。

它把LLM输出的token矩阵压缩成单个向量（768/1024/4096维），专门用来做语义检索和相似度计算。

LLM本身（如GPT、Claude）不做池化：

输入：100个token $\rightarrow$ 100×4096 矩阵

输出：100个位置的logits，每个位置预测下一个token是什么

=> 保留每个位置的独立表示，才能按顺序生成文本

Embedding模型 = LLM底座 + 池化头，池化这一步把 生成模型 变成了 表示模型。

LLM推理中的数据搬运

内存墙问题

- SRAM（静态随机存取存储器）：

速度极快（320GB/s单SM带宽，40TB/s级聚合带宽），但容量极小（每SM仅128KB），用于线程间高效交换数据

=> GPU芯片内的超高速小仓库

- HBM（高带宽内存）：

GPU显存的主力。2026年主流为HBM3e，带宽已达4.8TB/s（H200）到8TB/s（B200）。

=> HBM是芯片外的大仓库。

这是打破LLM推理显存墙的关键——大模型推理90%的时间花在数据搬运而非计算上

2026年的硬件选型逻辑已从 算力优先 转向 带宽优先+显存容量优先。
对于推理场景，显存带宽直接决定Token生成速度。

LLM推理中的数据搬运

在GPU中，线程间通过SRAM（即Shared Memory/共享内存）高效交换数据：

1. 数据复用（Tiling/分块计算）

矩阵乘法、卷积等计算中，多个线程需要重复读取同一块全局内存数据

先将数据从显存（HBM/GDDR）加载到SRAM，线程块内线程共享访问，避免重复读取（显存带宽仅~1TB/s，远低于SRAM的40TB/s聚合带宽）

2. 并行归约与协作计算

实现线程块内的归约操作（Reduction，如求和、求最大/最小值、矩阵行/列归一化）

前缀和（Prefix Sum/Scan）等需要线程间数据传递的算法

3. 结果聚合与缓存

卷积的多个输出通道复用同一输入特征图

Transformer中的Q/K/V矩阵计算时缓存中间结果

本质：用极低延迟，让协作线程以比访问显存快10-100倍的速度共享中间数据

LLM推理中的数据搬运

Thinking: 为什么LLM推理卡在数据搬运?

如果你在工厂加工（矩阵计算），但原材料（模型参数）存在10公里外的仓库（HBM）。

=> 虽然你加工速度极快，但卡车运货太慢，你90%时间在等货。

⇒ LLM几百GB参数无法全塞进SRAM，必须反复从HBM搬运到SM。

虽然SM计算非常快，但搬运速度比计算慢50倍以上

GPU算力被内存带宽卡死——这就是内存墙。

核心GPU型号

核心GPU型号

型号	架构	显存/带宽	定位	适用场景	说明
H100	Hopper	80GB HBM3 / 3.35TB/s	中流砥柱	7B-70B模型训练/推理，生态最成熟	交付稳定，云厂商主力
H200	Hopper	141GB HBM3e / 4.8TB/s	内存怪兽	长上下文推理 (>128K)、70B+模型单卡部署	解决"跑不动"问题的首选
B200	Blackwell	192GB HBM3e / 8TB/s	下一代主力	MoE架构、FP4低精度推理、千卡级集群	已大规模交付
B300	Blackwell Ultra	288GB HBM3e / 8TB/s	显存之王	超大模型单卡承载、边缘到云端统一	2025Q3已发布
L40S	Ada Lovelace	48GB GDDR6 / 864GB/s	推理黑马	放宽延迟预算的连续批处理场景	性价比之选
RTX 4090	Ada Lovelace	24GB GDDR6X	原型验证	1B-7B模型本地调试、教学用途	消费级，无NVLink

架构代际（从旧到新）：Ada Lovelace (2022) → Hopper (2022) → Blackwell (2025)。
先进程度：Blackwell最新，其次Hopper（数据中心专用），Ada Lovelace已属上一代。

选型建议

选型建议：

- <7B参数：用L40S或RTX 6000 Ada， H100性能过剩
- 7B-30B全参训练： H100（FP8+梯度检查点，生态最成熟）
- 30B-70B推理： H200（141GB显存可单卡运行70B，避免多卡通信开销）
- >70B dense或MoE（如DeepSeek-V3）： B200（192GB显存+NVLink 5，显著缓解显存瓶颈）
- 超长上下文(>128K)：训练选B200（高带宽），推理选H200（性价比更高）

显存需求:RTX 6000 Ada 大一倍（48GB），可本地运行 70B 模型（INT8），RTX4090 只能跑 30B 以下

NVLink 是 NVIDIA 的 GPU 间高速互联技术

B 系列（Blackwell）优势——生成式 AI 场景，H 系列（Hopper）仍有优势——科学计算场景

B 系列是专用 AI 芯片，H 系列是通用计算芯片。如果做大模型训练/推理，B200 明显更好；如果做气象模拟、分子动力学等科学计算，H100/H200 仍更优。

Blackwell架构的革命性变化

Blackwell架构的革命性变化：

- FP4精度支持：相比FP8，内存占用再减1.8倍，吞吐提升55%，且精度损失可接受
- 第五代NVLink：带宽1.8TB/s，适合千卡级超大规模训练
- 能效比：每次推理能耗仅为H100的1/25，Llama 3推理单卡可替代5个H100节点

资源配比

CPU与内存的隐形瓶颈：

- 推理不仅是GPU的事，也和CPU有关

CPU负责：并发请求调度、Tokenization（文本 $\leftrightarrow$ Token转换）、前后处理（JSON解析、正则过滤）

- CPU与GPU需要合理配比

若并发量大，CPU负载过高或系统内存（DRAM）不足 => GPU会陷入 饥饿等待 状态。

建议配比：每1张GPU配16-32核CPU + 512GB-1TB DRAM

功耗与散热

功耗与散热的考虑

- B200单卡功耗已达1000-1200W， B300将达1400W
- 2026年新建机房必须预留液冷基础设施，传统风冷已无法满足Blackwell集群需求

针对个人：

- 最多人买： RTX 4090（约¥15,000-20,000）

24GB 显存可跑 13B-30B 量化模型， FP8 支持成熟， 性价比优于 5090

- 预算紧张： 二手 RTX 3090（约¥5,500-6,000， 24GB 显存）

不推荐 5090： 价格贵 50%（约¥20,000-25,000）， 性能仅提升 15-20%

- 不推荐 RTX 6000 Ada： 价格高达 ¥46,000， 深度学习性能反而比 4090 慢 15%， 且为专业工作站设计

功耗与散热

针对个人GPU 散热情况（无需额外风扇）：

- RTX 4090 (450W)：标准三风扇风冷即可，满载约 62-70°C，普通 ATX 机箱足够
- RTX 5090 (575-600W)：风冷可压但温度较高（75-85°C），建议机箱风道优化（侧面进风、下进上出）；水冷效果更好（核心低 5°C，显存低 10°C）
- RTX 6000 Ada (300W)：涡轮散热设计，满载 85°C，噪音可控，适合紧凑机箱

只有多卡训练（2 张及以上）时才需要额外机箱风扇辅助散热，单卡使用无需担心

主流部署框架

从请求级调度 到 Token级调度

早期模型服务（如原生HuggingFace、NVIDIA Triton）采用请求级调度（Request Scheduling）

面对LLM变长输出时造成巨大算力浪费。

=> 现代LLM推理引擎的核心创新是Continuous Batching（连续批处理）和PD分离架构。

四大主流框架

框架	核心创新	适用场景	2026年最新动态	开源状态
vLLM	PagedAttention（页式KV Cache管理）	高吞吐、大批量Prompt的生产环境	V1架构重构（2025.1 alpha），加入PyTorch基金会（2025.5），生态最活跃	Apache 2.0
SGLang	RadixAttention（前缀树KV复用）	多轮对话、Agent/CoT/RAG等复杂交互	已部署超30万GPU，日处理数万亿Token，2025.3加入PyTorch生态	Apache 2.0
TensorRT-LLM	硬件原生优化、自定义CUDA核	追求极致性能的NVIDIA GPU环境	v1.0+推出trtllm-serve命令，原生支持OpenAI兼容API，门槛大幅降低	Apache 2.0
Ollama	极简封装（基于llama.cpp）	本地开发、边缘设备、CPU推理	定位不变，适合快速验证	Apache 2.0

框架选型

Thinking: 如何进行框架选型?

- 新手/快速验证 → Ollama (2分钟跑起来)
- 生产环境通用场景 → vLLM (生态最全, 文档最丰富)
- 长上下文/Agent/多轮对话 → SGLang (RadixAttention对重复前缀的复用率可达90%+)
- 极致性能/已用Triton → TensorRT-LLM (Blackwell上FP4推理必需)

Ollama本地部署大模型

Ollama使用

Ollama 官方主要支持 macOS 和 Linux，但 Windows用户也可以安装

方法1: 使用 WSL

Step1, 打开 PowerShell (管理员权限)，运行：

```
wsl --install
```

重启电脑后，WSL 会自动完成安装 (默认安装 Ubuntu)

Step2, 安装 Ollama

在 WSL 终端 (Ubuntu) 中运行：

```
curl -fsSL https://ollama.com/install.sh | sh
```

Step3, 启动 Ollama 服务

```
ollama serve
```

(保持此终端运行，另开一个新终端进行后续操作)

Ollama使用

方法 2: 直接下载 Windows 版

<https://ollama.com/>

安装后, Ollama 会作为服务运行 (可在任务管理器查看)

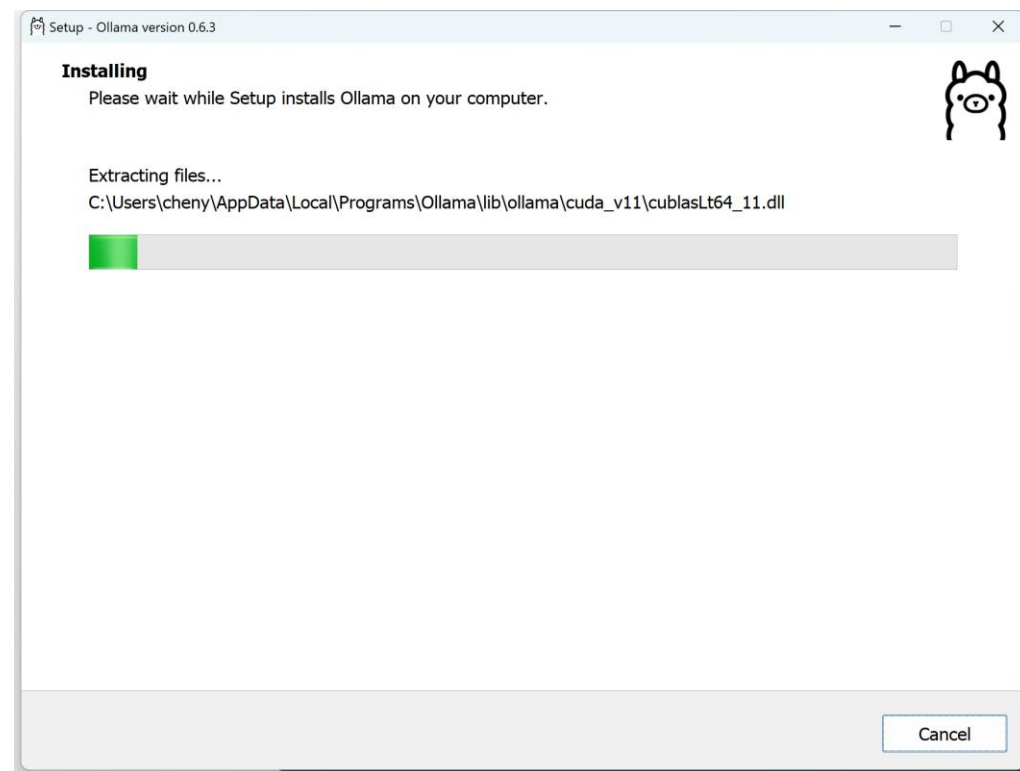


Get up and running with large language models.

Run Llama 3.3, DeepSeek-R1, Phi-4, Mistral, Gemma 3, and other models, locally.

Download ↓

Available for macOS,
Linux, and Windows



Ollama使用

Ollama官方模型库 <https://ollama.com/library>

下载 deepseek-r1:1.5b 模型

```
ollama pull deepseek-r1:1.5b
```

如果要删除该模型，可以使用

```
ollama rm deepseek-r1:1.5b
```

运行该模型，使用

```
ollama run deepseek-r1:1.5b
```

```
PS C:\Windows\System32> ollama run deepseek-r1:1.5b
>>> 你好啊
<think>

</think>
你好！很高兴见到你，有什么我可以帮忙的吗？
```

qwq

QwQ is the reasoning model of the Qwen series.

tools

32b

↓ 1.1M Pulls 🔖 8 Tags ⌚ Updated 2 weeks ago

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b

7b

8b

14b

32b

70b

671b

↓ 31.5M Pulls 🔖 29 Tags ⌚ Updated 7 weeks ago

CASE: Ollama使用 (使用Ollama REST API)

```
import requests

def query_ollama(prompt, model="deepseek-r1:1.5b"):
    url = "http://localhost:11434/api/generate"
    data = {
        "model": model,
        "prompt": prompt,
        "stream": False # 设置为 True 可以获取流式响应
    }

    response = requests.post(url, json=data)
    if response.status_code == 200:
        return response.json()["response"]
    else:
        raise Exception(f"API 请求失败: {response.text}")
```

使用示例

```
response = query_ollama("你好，请介绍一下你自己")
print(response)
```

<think>

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手 DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

</think>

您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手 DeepSeek-R1。如您有任何任何问题，我会尽我所能为您提供帮助。

Ollama 本身提供 REST API 接口，默认运行在 localhost:11434 运行 ollama serve 时，API 服务会自动启动。

CASE: Ollama使用 (FastAPI封装)

如果想添加额外的功能或自定义 API 端点, 可以使用 FastAPI 创建一个包装层:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import requests

app = FastAPI()

# 定义请求模型
class ChatRequest(BaseModel):
    prompt: str
    model: str = "deepseek-r1:1.5b"

# 允许跨域请求 (根据需要配置)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
@app.post("/api/chat")
async def chat(request: ChatRequest):
    ollama_url = "http://localhost:11434/api/generate"
    data = {
        "model": request.model,
        "prompt": request.prompt,
        "stream": False
    }

    response = requests.post(ollama_url, json=data)
    if response.status_code == 200:
        return {"response": response.json()["response"]}
    else:
        return {"error": "Failed to get response from Ollama"}, 500

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

7-ollama-fastapi.py

CASE：Ollama使用（FastAPI封装）

创建好API服务后，需要启动

python 7-ollama-fastapi.py

```
INFO: Started server process [34680]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

创建Python 客户端调用启动好的FastAPI（8000端口）

```
import requests

response = requests.post(
    "http://localhost:8000/api/chat",
    json={"prompt": "你好，请介绍一下你自己"}
)

print(response.json())
```

```
{'response': '<think>\n嗯，用户刚问我“你好”，然后让我介绍一下自己。看起来他可能是在测试我的响应，或者想了解我的身份。\\n\\n我应该先回应问候，接着详细介绍自己的身份和背景。要保持回答专业且全面，包括教育背景、工作经历和个人爱好。\\n\\n考虑到我是通过自然语言处理来思考这个问题的，我可以先列出一些关键点：教育背景、工作经验、兴趣爱好以及未来的职业规划。\\n\\n这样用户不仅得到了一个完整的回答，还能了解我为什么会选择这些信息，并且可能展示出我对自己的理解和自信。\\n\\n最后，用礼貌的结束语让用户放心。\\n</think>\n\n您好！我是由中国的深度求索（DeepSeek）公司开发的智能助手DeepSeek-R1。我是一个来自中国的人工智能助手，专注于帮助您解答各种问题。我的专业背景是计算机科学和人工智能，具备丰富的知识体系和解决问题的能力。如果您有任何疑问或需要进一步的帮助，请随时告诉我。'}
```

7-ollama-fastapi-python客户端.py

Ollama 推理引擎

Ollama 推理引擎：基于 llama.cpp，C/C++ 编写的高性能推理库，针对消费级 GPU 和 CPU 优化

Thinking: 什么是 llama.cpp?

llama.cpp 是开源的高性能LLM推理引擎，纯C/C++实现，主打跨平台本地部署。

支持GGUF格式量化（4/8bit压缩），让消费级CPU甚至手机也能流畅运行大模型。

零依赖、可离线使用，被Ollama、LM Studio等工具广泛采用，是本地LLM生态的核心基础设施。

llama.cpp 是软件名称，也是 GitHub 上的开源项目名称。 <https://github.com/ggml-org/llama.cpp>

.cpp 是 C++ 的源代码文件后缀，它明确用 .cpp 后缀告诉开发者：这是一个用 C++ 语言编写的项目

Ollama API服务

实际上Ollama是双层架构：

- Go 层：写 API 服务、模型管理、命令行交互（利用 Go 的并发和网络优势）；
- llama.cpp (C++)：通过 CGO 封装为 Go 可调用的接口，负责底层矩阵计算和模型推理。

=> Go 管调度和服务，C++ 管算力，两者通过 CGO 无缝协作。

Go 语言是 Google 开发的静态类型语言，主打高并发（goroutine 轻量线程）、编译速度快和内存管理自动化。语法简洁，自带垃圾回收，适合写网络服务和云原生工具。

Ollama的局限性

Ollama的局限性：不支持Continuous Batching，并发能力差，仅适合单用户调试

Thinking：什么是Continuous Batching 持续批处理？

传统批处理（Static/Dynamic Batching）要求整批请求同步开始、同步结束。而 Continuous Batching 将粒度细化到 Token 级别

传统批处理： [请求A 请求B 请求C] → 等待全部完成 → [请求D 请求E ...]
└────────── 必须等待最慢的请求 ─────────┘

Continuous： 迭代1： [A₁, B₁, C₁]
 迭代2： [A₂, B₂, C₂]
 迭代3： [A₃, B₂, 新请求D₁] ← 请求C完成，D立即插入
 迭代4： [A₄, B₃, D₂, 新请求E₁] ← 请求B完成，E立即插入

Ollama的局限性

Thinking: Ollama的API层不是通过GO语言来实现的么？怎么还不支持并发了？

Go 的并发≠推理层的连续批处理（Continuous Batching）。

Go 层确实能并发处理 HTTP 请求，但瓶颈在底层推理引擎 llama.cpp：

Continuous Batching 是推理优化技术：多个用户的请求在同一个推理步骤中动态拼接，共享一次 forward pass。这能大幅提升 GPU 利用率。

llama.cpp 的限制：它主要为单用户本地运行设计，缺乏 vLLM 那样的 Continuous Batching 和 PagedAttention 机制。即使 Go 层并发接收了 10 个请求，llama.cpp 可能还是串行执行，或简单静态 batching，导致 GPU 利用率低。

Ollama 的定位：本地单用户调试工具，不是生产级服务。

Go 负责快速启动和管理，但不改造 llama.cpp 的推理内核。

vllm高效部署

vllm使用

vllm使用：是由伯克利大学 LMSYS 组织开源的LLM高速推理框架，用于提升LLM的吞吐量与内存使用效率。它通过 PagedAttention 技术高效管理注意力键和值的内存，并结合连续批处理技术优化推理性能。vLLM 支持量化技术、分布式推理、与 Hugging Face 模型无缝集成等功能

```
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B --tensor-parallel-size 2 --max-model-len 32768 --enforce-eager
```

- vllm serve, 启动 vLLM 推理服务的命令
- deepseek-ai/DeepSeek-R1-Distill-Qwen-32B, Hugging Face 模型库中的模型名称, vLLM 会尝试从 HF 下载模型
- --tensor-parallel-size 2, 启用张量并行, 在 2 个 GPU 上分布式运行模型 (适合 32B 大模型)
- --max-model-len 32768, 设置模型的最大上下文长度 (32K tokens), 确保能处理长文本。
- --enforce-eager, 禁用 CUDA Graph 优化 (可能在某些环境下更稳定, 但性能稍低)

Vllm使用

Thinking: 如果我在本地的ubuntu下面有 /root/autodl-tmp/models/tcl90/deepseek-r1-distill-qwen-32b-gptq-int4, 如何使用vllm进行推理?

```
vllm serve /root/autodl-tmp/models/tcl90/deepseek-r1-distill-qwen-32b-gptq-int4 --tensor-parallel-size 1 --max-model-len 32768 --enforce-eager --quantization gptq --dtype half
```

关键改动: 指定本地路径: 替换 HF 模型名为你的本地路径。

--quantization gptq: 显式声明使用 GPTQ 量化。

--dtype: 设为 half (FP16) 或 auto (自动选择), 因为 GPTQ 本身是 4-bit, 但计算时需指定中间精度。

```
Loading safetensors checkpoint shards: 0% Completed | 0/4 [00:00<?, ?it/s]
Loading safetensors checkpoint shards: 25% Completed | 1/4 [00:00<00:02, 1.13it/s]
Loading safetensors checkpoint shards: 50% Completed | 2/4 [00:02<00:02, 1.22s/it]
Loading safetensors checkpoint shards: 75% Completed | 3/4 [00:03<00:01, 1.22s/it]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:04<00:00, 1.17s/it]
Loading safetensors checkpoint shards: 100% Completed | 4/4 [00:04<00:00, 1.17s/it]
```

```
ERROR 03-25 11:02:05 [core.py:340] RuntimeError: CUDA out of memory occurred when warming up sampler
ase try lowering `max_num_seqs` or `gpu_memory_utilization` when initializing the engine.
ERROR 03-25 11:02:05 [core.py:340]
CRITICAL 03-25 11:02:05 [core_client.py:269] Got fatal signal from worker processes, shutting down. S
cause issue.
```

Vllm使用

```
vllm serve /root/autodl-tmp/models/tclf90/deepseek-r1-distill-qwen-32b-gptq-int4 --tensor-parallel-size 1 --max-model-len 4096 --quantization gptq --dtype half --gpu-memory-utilization 0.8 --max-num-seqs 8 --enforce-eager
```

```
INFO 03-25 11:06:45 [loader.py:429] Loading weights took 4.84 seconds
INFO 03-25 11:06:45 [gpu_model_runner.py:1176] Model loading took 18.1678 GB and 5.752492 seconds
ERROR 03-25 11:06:47 [core.py:340] EngineCore hit an exception: Traceback (most recent call last):
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 332, in run
_engine_core
ERROR 03-25 11:06:47 [core.py:340]     engine_core = EngineCoreProc(*args, **kwargs)
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 287, in __i
nit__
ERROR 03-25 11:06:47 [core.py:340]     super().__init__(vllm_config, executor_class, log_stats)
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 62, in __in
it__
ERROR 03-25 11:06:47 [core.py:340]     num_gpu_blocks, num_cpu_blocks = self._initialize_kv_caches(
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/engine/core.py", line 124, in _in
italize_kv_caches
ERROR 03-25 11:06:47 [core.py:340]     kv_cache_configs = get_kv_cache_configs(vllm_config, kv_cache_specs,
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/core/kv_cache_utils.py", line 576
, in get_kv_cache_configs
ERROR 03-25 11:06:47 [core.py:340]     check_enough_kv_cache_memory(vllm_config, kv_cache_spec,
ERROR 03-25 11:06:47 [core.py:340]   File "/root/miniconda3/lib/python3.10/site-packages/vllm/v1/core/kv_cache_utils.py", line 468
, in check_enough_kv_cache_memory
ERROR 03-25 11:06:47 [core.py:340]     raise ValueError("No available memory for the cache blocks. ")
ERROR 03-25 11:06:47 [core.py:340] ValueError: No available memory for the cache blocks. Try increasing `gpu_memory_utilization` w
hen initializing the engine.
ERROR 03-25 11:06:47 [core.py:340]
CRITICAL 03-25 11:06:47 [core_client.py:269] Got fatal signal from worker processes, shutting down. See stack trace above for root
cause issue.
Killed
```

尽管我们已经将参数调整到非常保守的配置
(max-model-len=4096、gpu-memory-
utilization=0.8)，但 32B GPTQ 量化模型仍然
无法在 24GB GPU 上运行。

Vllm使用

Thinking: 在 autodl 上系统盘有限, 很容易满, 如何将 pip install 安装的包, 放到 autodl-tmp 中?

在数据盘创建虚拟环境

```
python -m venv /root/autodl-tmp/venv
```

激活 (每次重启或新终端都要执行)

```
source /root/autodl-tmp/venv/bin/activate
```

升级 pip 后安装

```
pip install --upgrade pip
```

```
pip install vllm transformers --upgrade
```

查看占用空间: `du -sh /root/autodl-tmp/venv`

```
105/174 [numba]
Successfully installed MarkupSafe-3.0.3 aiohappyeyeballs-2.6.1 aiohttp-3.13.3 aiosignal-1.4.0 annotated-doc-0.0.4 annotate
d-types-0.7.0 anthropic-0.86.0 anyio-4.12.1 apache-tvm-ffi-0.1.9 astor-0.8.1 async-timeout-5.0.1 attrs-26.1.0 blake3-1.0.8
cachetools-7.0.5 cbor2-5.8.0 certifi-2026.2.25 cffi-2.0.0 charset-normalizer-3.4.6 click-8.3.1 cloudpickle-3.1.2 compress
ed-tensors-0.13.0 cryptography-46.0.5 cuda-bindings-12.9.4 cuda-pathfinder-1.4.3 cuda-python-12.9.4 depyf-0.20.0 dill-0.4
.1 diskcache-5.6.3 distro-1.9.0 dnspython-2.8.0 docstring-parser-0.17.0 einops-0.8.2 email-validator-2.3.0 exceptiongroup-1
.3.1 fastapi-0.135.1 fastapi-cli-0.0.24 fastapi-cloud-cli-0.15.0 fastar-0.9.0 filelock-3.25.2 flashinfer-python-0.6.6 froz
enlist-1.8.0 fsspec-2026.2.0 gguf-0.18.0 googleapis-common-protos-1.73.0 grpcio-1.78.0 h11-0.16.0 hf-xet-1.4.2 httpcore-1
.0.9 httptools-0.7.1 httpx-0.28.1 httpx-sse-0.4.3 huggingface-hub-0.36.2 idna-3.11 ijson-3.5.0 importlib-metadata-8.7.1 int
eregular-0.3.3 jinja2-3.1.6 jiter-0.13.0 jmespath-1.1.0 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 lark-1.2.2 ll
guidance-1.3.0 llvmlite-0.44.0 lm-format-enforcer-0.11.3 loguru-0.7.3 markdown-it-py-4.0.0 mcp-1.26.0 mdurl-0.1.2 mistral_
common-1.10.0 model-hosting-container-standards-0.1.14 mpmath-1.3.0 msgspec-0.20.0 multidict-6.7.1 networkx-3.4.2 ninja-1
.13.0 numba-0.61.2 numpy-2.2.6 nvidia-cublas-cul2-12.8.4.1 nvidia-cuda-cupti-cul2-12.8.90 nvidia-cuda-nvrtc-cul2-12.8.93 nv
idia-cuda-runtime-cul2-12.8.90 nvidia-cudnn-cul2-9.10.2.21 nvidia-cudnn-fronten-1.18.0 nvidia-cufft-cul2-11.3.3.83 nvidia
-cufile-cul2-1.13.1.3 nvidia-curand-cul2-10.3.9.90 nvidia-cusolver-cul2-11.7.3.90 nvidia-cuspars-cul2-12.5.8.93 nvidia-cu
sparselt-cul2-0.7.1 nvidia-cutlass-dsl-4.4.2 nvidia-cutlass-dsl-libs-base-4.4.2 nvidia-ml-py-13.595.45 nvidia-nccl-cul2-2
.27.5 nvidia-nvjitlink-cul2-12.8.93 nvidia-nvshmem-cul2-3.4.5 nvidia-nvtx-cul2-12.8.90 openai-2.24.0 openai-harmony-0.0.8 o
pencv-python-headless-4.13.0.92 opentelemetry-api-1.40.0 opentelemetry-exporter-otlp-1.40.0 opentelemetry-exporter-otlp-pr
oto-common-1.40.0 opentelemetry-exporter-otlp-proto-grpc-1.40.0 opentelemetry-exporter-otlp-proto-http-1.40.0 opentelemetr
y-proto-1.40.0 opentelemetry-sdk-1.40.0 opentelemetry-semantic-conventions-0.61b0 opentelemetry-semantic-conventions-ai-0
.5.0 outlines_core-0.2.11 packaging-26.0 partial-json-parser-0.2.1.1.post7 pillow-12.1.1 prometheus-fastapi-instrumentator
-7.1.0 prometheus-client-0.24.1 propcache-0.4.1 protobuf-6.33.6 psutil-7.2.2 py-cpuinfo-9.0.0 pybase64-1.4.3 pycountry-26.2
.16 pycparser-3.0 pydantic-2.12.5 pydantic-core-2.41.5 pydantic-extra-types-2.11.1 pydantic-settings-2.13.1 pygments-2.19
.2 pyjwt-2.12.1 python-dotenv-1.2.2 python-json-logger-4.0.0 python-multipart-0.0.22 pyyaml-6.0.3 pyzmq-27.1.0 quack-kernel
s-0.3.4 referencing-0.37.0 regex-2026.2.28 requests-2.32.5 rich-14.3.3 rich-toolkit-0.19.7 rignore-0.7.6 rpds-py-0.30.0 sa
fetensors-0.7.0 sentencepiece-0.2.1 sentry-sdk-2.55.0 setproctitle-1.3.7 shellingham-1.5.0 sniffio-1.3.1 sse-starlette-3.3
.3 starlette-0.52.1 supervisor-4.3.0 sympy-1.14.0 tabulate-0.10.0 tiktoken-0.12.0 tokenizers-0.22.2 tomli-2.4.0 torch-2.10
.0 torch-c-dlpack-ext-0.1.5 torchaudio-2.10.0 torchvision-0.25.0 tqdm-4.67.3 transformers-4.57.6 triton-3.6.0 typer-0.24.1
typing-inspection-0.4.2 typing_extensions-4.15.0 urllib3-2.6.3 uvicorn-0.42.0 uvloop-0.22.1 vllm-0.18.0 watchfiles-1.1.1
websockets-16.0 xgrammar-0.1.32 yarll-1.23.0 zipp-3.23.0
(venv) (base) root@autodl-container-85234cba89-6125e20f:~/autodl-tmp/models/gwen#
```

Vllm使用

Thinking: 如何查看某个 python包的版本, 比如 torch?

```
pip show torch
```

```
(venv) (base) root@autodl-container-85234cba89-6125e20f:~/autodl-tmp/models/qwen# pip show torch
Name: torch
Version: 2.10.0
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
Home-page: https://pytorch.org
Author:
Author-email: PyTorch Team <packages@pytorch.org>
License: BSD-3-Clause
Location: /root/autodl-tmp/venv/lib/python3.10/site-packages
Requires: cuda-bindings, filelock, fsspec, jinja2, networkx, nvidia-cublas-cu12, nvidia-cuda-cupti-cu12, nvidia-cuda-runtime-cu12, nvidia-cudnn-cu12, nvidia-cufft-cu12, nvidia-cufile-cu12, nvidia-ml-cu12, nvidia-cusparselt-cu12, nvidia-nccl-cu12, nvidia-nvjitlink-cu12, nvidia-nvtx-cu12, sympy, triton, typing-extensions
Required-by: compressed-tensors, flashinfer-python, quack-kernels, torch_c_dlpack_ext, torchaudio, mmcv
```

如果后续运行报错, 可以考虑使用
PyTorch 2.6.0

vLLM 0.18.0 官方推荐 PyTorch 2.6.0

```
pip install torch==2.6.0 torchvision
torchaudio --upgrade
```

Vllm使用

如果你重新安装了PyTorch 2.6.0, 可能需要强制重新安装 vLLM, 让它针对 PyTorch 2.6.0 重新编译 C++ 扩展:

1. 先卸载 vllm (不卸载依赖)

```
pip uninstall vllm -y
```

2. 强制重新安装并重新编译 (约 5-10 分钟)

```
pip install vllm --force-reinstall --no-cache-dir
```

vllm使用

启动模型（重要参数）

Qwen3 用 vLLM 0.18.0 默认的 V1 引擎 + torch.compile 会报错（FakeTensorMode 相关），必须禁用编译优化：

```
vllm serve /root/autodl-tmp/models/qwen/Qwen3-0.6B \  
--enforce-eager \  
--gpu-memory-utilization 0.9
```

核心参数：

--enforce-eager：必须加，禁用 torch.compile，否则报 AttributeError: FakeTensorMode

--gpu-memory-utilization：控制显存占用，默认 0.9 即可

vllm使用

```
odel's `generation_config.json`: `{ 'temperature': 0.6, 'top_k': 20, 'top_p': 0.95}`. If this is not intended, please relau
nch vLLM instance with `--generation-config vllm`.
(APIServer pid=7722) INFO 03-21 14:20:58 [hf.py:320] Detected the chat template content format to be 'string'. You can set
`--chat-template-content-format` to override this.
(APIServer pid=7722) INFO 03-21 14:20:58 [api_server.py:580] Starting vLLM server on http://0.0.0.0:8000
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:37] Available routes are:
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /openapi.json, Methods: HEAD, GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /docs, Methods: HEAD, GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /docs/oauth2-redirect, Methods: HEAD, GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /redoc, Methods: HEAD, GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /tokenize, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /detokenize, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /load, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /version, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /health, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /metrics, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/models, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /ping, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /ping, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /invocations, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/chat/completions, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/responses, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/responses/{response_id}, Methods: GET
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/responses/{response_id}/cancel, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/completions, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/messages, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/messages/count_tokens, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /inference/v1/generate, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /scale_elastic_ep, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /is_scaling_elastic_ep, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/chat/completions/render, Methods: POST
(APIServer pid=7722) INFO 03-21 14:20:58 [launcher.py:46] Route: /v1/completions/render, Methods: POST
(APIServer pid=7722) INFO: Started server process [7722]
(APIServer pid=7722) INFO: Waiting for application startup.
(APIServer pid=7722) INFO: Application startup complete.
```

说明 vllm 已经启动起来了

新开终端测试:

curl http://localhost:8000/v1/models

```
(base) root@autodl-container-85234cba89-6125e20f:~/autodl-tmp/models/qwen# curl http://localhost:8000/v1
/models
{"object": "list", "data": [{"id": "/root/autodl-tmp/models/qwen/Qwen3-0.6B", "object": "model", "created": 1774
074320, "own": "vllm", "root": "/root/autodl-tmp/models/qwen/Qwen3-0.6B", "parent": null, "max_model_len": 40960, "
permission": [{"id": "mo-81c17f444f599757", "object": "model_permission", "created": 1774074320, "allow_create_
engine": false, "allow_sampling": true, "allow_logprobs": true, "allow_search_indices": false, "allow_view": true, "allow
_fine_tuning": false, "organization": "**", "grou
(base) root@autodl-container-85234cba89-6125e20f:~/autodl-tmp
```

方法1: curl 直接调用

Thinking: VLLM启动好了, 如何使用?

方法1: curl 直接调用 (最快测试)

```
curl http://localhost:8000/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "/root/autodl-tmp/models/qwen/Qwen3-0.6B",  
  "messages": [{"role": "user", "content": "你好, 请介绍下自己"}],  
  "max_tokens": 512  
'
```

```
(base) root@autodl-container-85234cba89-6125e20f:~/autodl-tmp/models/qwen# curl http://localhost:8000/v1/  
chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "/root/autodl-tmp/models/qwen/Qwen3-0.6B",  
  "messages": [{"role": "user", "content": "你好, 请介绍下自己"}],  
  "max_tokens": 512  
}'  
{  
  "id": "chatcmpl-a3a3ae97045daaa8",  
  "object": "chat.completion",  
  "created": 1774074524,  
  "model": "/root/autodl-tmp/models/qwen/Qwen3-0.6B",  
  "choices": [{"index": 0, "message": {"role": "assistant", "content": "<think>\n好的, 用户发来一条消息: '你好, 请介绍下自己'。首先, 我需要确认用户的身份和意图。用户可能是在测试我的反应, 或者想了解我的基本信息。我应该保持友好和积极的态度, 同时提供一些有用的信息。\\n\\n接下来, 我要考虑如何自然地回应。用户可能希望得到一个简短而友好的介绍, 同时保持对话的流畅。我可以用一些表情符号来增加亲切感, 比如 '😊' 来表示友好。\\n\\n另外, 用户可能没有说出来的深层需求是什么? 他们可能希望了解我的技能或兴趣, 或者只是想测试我的反应。因此, 我需要确保回答既符合用户的需求, 又不会显得生硬或不专业。\\n\\n还要注意语气的友好和真诚, 避免使用过于正式或机械的表达。同时, 保持回答的简洁, 避免冗长, 这样用户能轻松阅读。\\n\\n最后, 检查是否有需要调整的地方, 比如是否需要添加更多细节或询问用户是否需要进一步帮助。确保整个回应符合我的角色设定, 同时保持自然和真诚。\\n</think>\n\\n😊你好呀! 我是你的AI助手, 很高兴和你交流! 有什么问题或需要帮助吗? 😊"}],  
  "refusal": null,  
  "annotations": null,  
  "audio": null,  
  "function_call": null,  
  "tool_calls": [],  
  "reasoning": null,  
  "logprobs": null,  
  "finish_reason": "stop",  
  "stop_reason": null,  
  "token_ids": null,  
  "service_tier": null,  
  "system_fingerprint": null,  
  "usage": {"prompt_tokens": 13, "total_tokens": 253, "completion_tokens": 240, "prompt_tokens_details": null},  
  "prompt_logprobs": null,  
  "prompt_token_ids": null,  
  "kv_transfer_params": null  
}(base) root@autodl-contai  
ner-85234cba89-6125e20f:~/autodl-tmp/models/qwen#
```

方法1: curl 直接调用

Qwen3 特殊功能 - 控制思考模式:

开启思考模式 (模型会先思考再回答)

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "/root/autodl-tmp/models/qwen/Qwen3-0.6B",
  "messages": [{"role": "user", "content": "解方程  $2x + 3 = 7$ "}],
  "max_tokens": 1024,
  "extra_body": {"enable_thinking": true}
}'
```

```
{
  "id": "chatcmpl-a9e6ba2e6251ec82",
  "object": "chat.completion",
  "created": 1774074605,
  "model": "/root/autodl-tmp/models/qwen/Qwen3-0.6B",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "<think>\n嗯，我现在要解这个方程 $2x+3=7$ 。好，先回忆一下解方程的基本步骤。首先，我应该把方程中的未知数 $x$ 单独留在左边，对吧？所以，我需要把3移到等号右边，这样两边都减去3，对吧？不过，先别急，让我仔细想想。\\n\\n首先，原方程是 $2x + 3 = 7$ 。我需要解 $x$ ，所以我要把3用减法来抵消。也就是说，两边同时减去3，这样左边剩下的是 $2x$ ，右边是 $7-3$ ，也就是4。所以方程变成 $2x = 4$ 。这时候， $x$ 应该是4除以2，也就是2。对吗？\\n\\n不过，我应该再检查一下，确保自己没有哪里弄错了。代入原方程，左边是2乘以2加上3，等于4加3，也就是7，右边也是7，所以是对的。看起来没问题。\\n\\n不过，有时候可能会有其他方法解这个方程，比如先移项，或者用其他代数技巧，不过我觉得这样已经是最简单的了。有没有可能哪里漏掉了什么？比如说，有没有可能把方程写错了？题目是 $2x + 3 = 7$ ，没错，所以步骤是对的。\\n\\n再想一下，有没有可能把加号理解错了？比如，是不是应该是 $2x + 3$ 等于7，或者有没有可能把3放在另一个位置？不过题目已经明确写成 $2x + 3 = 7$ ，所以应该没问题。\\n\\n总结一下，解题过程应该是：两边减3，得到 $2x = 4$ ，然后两边除以2，得到 $x = 2$ 。代入原方程验证正确，所以答案应该是 $x=2$ 。\\n\\n嗯，看来自己都没问题，应该没错吧。不过为了彻底确认，再用另一种方法试试。比如，把方程看成 $2x = 7 - 3$ ，也就是 $2x = 4$ ，同样得到 $x=2$ 。没错，两种方法都是一样的结果。所以应该没问题了。\\n\\n总之，解这个方程得到的 $x$ 是2。\\n</think>\\n\\n解方程  $2x + 3 = 7$  的步骤如下：\\n\\n1. **移项**：将方程中的常数项  $3$  移到等号右边，得到  $2x = 7 - 3$ 。\\n2. **化简**：计算右边， $7 - 3 = 4$ ，因此方程变为  $2x = 4$ 。\\n3. **求解**：两边同时除以2，得到  $x = \\frac{4}{2} = 2$ 。\\n\\n**验证**：将  $x = 2$  代入原方程，左边为  $2 \\times 2 + 3 = 4 + 3 = 7$ ，右边也为7，符合方程。\\n\\n**最终答案**：  $x = 2$ 。",
        "refusal": null,
        "annotations": null,
        "audio": null,
        "function_call": null,
        "tool_calls": [],
        "reasoning": null,
        "logprobs": null,
        "finish_reason": "stop",
        "stop_reason": null,
        "token_ids": null
      },
      "service_tier": null,
      "system_fingerprint": null,
      "usage": {
        "prompt_tokens": 20,
        "total_tokens": 645,
        "completion_tokens": 625,
        "prompt_tokens_details": null,
        "prompt_logprobs": null,
        "prompt_token_ids": null,
        "kv_transfer_params": null
      }
    }
  ]
}
```


方法2：Python调用

方法2：Python 调用

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="dummy" # vLLM 不验证 key, 但必须填
)

response = client.chat.completions.create(
    model="/root/autodl-tmp/models/qwen/Qwen3-0.6B",
    messages=[{"role": "user", "content": "你好，请介绍下自己"}],
    max_tokens=512
)

print(response.choices[0].message.content)
```

<think>

好的，用户发来消息：“你好，请介绍下自己”。我需要先友好地回应，然后介绍自己的身份和能力。用户可能希望了解我的功能或背景，所以应该简明扼要地说明。同时，保持回答的自然和亲切，避免使用过于正式或机械的表达。另外，要注意用户的潜在需求，比如他们可能在使用我的服务时需要更多的信息，或者想确认我的能力范围。最后，确保回答简洁，让用户觉得轻松易懂。

</think>

您好！我是您的助手，可以为您提供帮助和支持。有什么我可以帮您的吗？😊

1-qwen3-vllm.py

Summary

查看当前虚拟环境中的所有相关包的版本：

先确保激活了 venv，然后执行：

```
source /root/autodl-tmp/venv/bin/activate
```

```
pip list | grep -E
```

```
"(vllm|torch|transformer|openai|httpx|nvidia|flash|numpy|fastapi|uvicorn|pydantic|tokenizers|sentencepiece)"
```

```
fastapi-cloud-cli          0.15.0
flashinfer-python          0.6.6
httpx                       0.28.1
httpx-sse                   0.4.3
numpy                       2.2.6
nvidia-cublas-cu12         12.8.4.1
nvidia-cuda-cupti-cu12     12.8.90
nvidia-cuda-nvrtc-cu12     12.8.93
nvidia-cuda-runtime-cu12   12.8.90
nvidia-cudnn-cu12          9.10.2.21
nvidia-cudnn-frontendl     1.18.0
nvidia-cufft-cu12          11.3.3.83
nvidia-cufile-cu12         1.13.1.3
nvidia-curand-cu12         10.3.9.90
nvidia-cusolver-cu12       11.7.3.90
nvidia-cuspars-cu12        12.5.8.93
nvidia-cusparselt-cu12     0.7.1
nvidia-cutlass-dsl         4.4.2
nvidia-cutlass-dsl-lib-base 4.4.2
nvidia-ml-py               13.595.45
nvidia-nccl-cu12           2.27.5
nvidia-nvjitlink-cu12      12.8.93
nvidia-nvshmem-cu12        3.4.5
nvidia-nvtx-cu12           12.8.90
openai                     2.24.0
openai-harmony              0.0.8
prometheus-fastapi-instrumentator 7.1.0
pydantic                   2.12.5
pydantic_core               2.41.5
pydantic-extra-types        2.11.1
pydantic-settings           2.13.1
sentencepiece               0.2.1
tokenizers                  0.22.2
torch                       2.10.0
torch-c-dlpack-ext         0.1.5
torchaudio                  2.10.0
```

Summary

Python工具箱版本说明：

包	推荐版本	容错说明
vllm	0.18.0	唯一必需版本，不可变
torch	2.6.0	不可变，2.5 会报错
httpx	0.27.2	不可变，0.28+ 导致 Python 无法调用
transformers	4.57.6 或 5.3.0	可灵活，4.x 无警告，5.x 有警告但可用
openai	1.x+	较宽松，1.20+ 即可

SGLang框架

SGLang框架

SGLang 框架:

开源的大模型推理引擎，由加州大学伯克利分校的 LMSYS 团队开发（和 Vicuna、Chatbot Arena 是同一个团队），2025 年刚被 PyTorch 官方收编为生态项目。

=> 定位是 更快的 vLLM，特别适合做高并发、低延迟的 API 服务。

性能特点:

- RadixAttention: 智能前缀缓存，多轮对话场景下能把重复计算的 Prompt 缓存复用，长上下文吞吐量是 vLLM 的 2-5 倍
- 结构化输出快 10 倍: 生成 JSON、函数调用参数时，比标准引擎快一个数量级
- SGLang Runtime: 原生支持多模态（图片+视频理解），DeepSeek R1 等推理模型开箱即用
- Pythonic DSL: 写 LLM 应用像写普通 Python 代码一样，支持条件判断、循环、并行调用

应用场景

性能对比（H100 单卡）

- **吞吐量：**比 vLLM 高 29%（约 16,200 vs 12,500 tok/s）
- **成本：**百万日活场景下，每月可节省约 \$15,000 GPU 费用
- **首 Token 延迟：**配合 PD 分离技术，可降低 60%

适合什么场景：

- **聊天机器人/客服：**多轮对话多的场景，前缀缓存命中率可达 75-95%
- **Agent/Function Calling：**需要频繁输出结构化 JSON 的场景
- **长文档处理：**论文总结、代码审查等长上下文任务
- **多模态 API：**同时处理图文视频的复杂请求

框架选择

- vLLM: 最主流

互联网大厂 API 服务、阿里云/腾讯云 GPU 实例内置、绝大多数 AI 初创公司首选

- SGLang: 快速增长

xAI (Grok)、Microsoft Azure、阿里盘古等大规模部署；2025年3月刚加入 PyTorch 生态

- Ollama: 个人/边缘端

本地开发、教育、边缘设备，不适合高并发 API 服务

企业大模型API部署

Thinking: 如果你想在企业内部搭建OpenClaw, 做 API 服务选哪个?

稳妥选择: vLLM

- PagedAttention: 显存利用率从 30% 提升到 90%+, 支持千级并发
- 生态最成熟: 支持模型最多 (几乎所有 HuggingFace 格式), 文档最全, GitHub Issues 里有现成答案
- 生产验证: 国内头部大模型厂商的 API 服务很多基于 vLLM 改造
- 缺点: 在 H100 上的吞吐量 (~12,500 tok/s) 比 SGLang 低约 29%

企业大模型API部署

性能优先：SGLang

- 如果你的团队有较强的底层优化能力，且追求极致性价比（省 GPU = 省钱），选 SGLang:
- 吞吐量更高：H100 上可达 16,200 tok/s，比 vLLM 快 29%，日活百万级请求下每月可省约 \$15,000 GPU 成本
- RadixAttention：对多轮对话、Agent 场景优化极好，前缀缓存命中率 75-95%，长上下文吞吐量可达 vLLM 的 5 倍
- 结构化输出快：JSON 生成速度比 vLLM 快 10 倍，适合 Function Calling
- 现状：生态在快速追赶（已支持 DeepSeek V3/R1、AMD GPU、昇腾 910B），但文档和社区不如 vLLM 成熟

打卡：大模型部署



对开源大模型进行私有化部署，选择适合的大模型，比如：

- deepseek-r1:1.5b, 7b, 8b, 14b, 32b, 70b等
- 或者qwen3系列： 0.6b, 1.7b, 4b, 8b, 14b, 30b, 32b等

选择适合的部署方法

- Vllm高效部署
- Ollama部署
- SGLang部署

部署任意的模型，选择其中一种部署的方法，调通即可，比如使用 ollama部署qwen3-0.6b

结构化信息 Outlines

结构化信息的好处

Thinking: 生成式AI如果输出结构化的信息，比如JSON格式，有什么好处？

标准化： 输出一致的格式。

可读性： 易于理解和处理。

易于解析： 很多语言都能轻松解析JSON格式，便于处理。

生成JSON结构化信息有多种场景：

客户信息管理： 创建结构化客户档案，包括姓名、联系方式、车辆信息、保险类型等，便于管理和使用。

索赔处理： 生成索赔申请的结构化数据，包含事故描述、损失估算、相关文件等，以简化索赔审核流程。

保单生成： 自动生成保单信息的JSON格式，便于系统记录和客户查看，确保所有条款清晰可见。

Outlines工具

Outlines工具:

- 开源的 Python 库，为LLM提供结构化文本生成能力

<https://github.com/dottxt-ai/outlines>

- 支持多种语言模型，如 OpenAI、Transformers、llama.cpp、exllama2 等
- 与非结构化生成相比，几乎不增加额外开销。
- 灵活性高，支持与循环、条件语句和自定义 Python 函数结合使用

- 支持多种结构化生成方式

比如多项选择、类型约束、正则表达式结构化生成、JSON 生成以及基于上下文无关文法的生成。比如：

正则表达式：email地址、手机号

Json: {"name": "mingming", "Age": }

Thinking: 什么是非结构化生成？

指 LLM 自由发挥生成文本，没有格式约束。

结构化生成（Outlines 做的）是强制模型按 JSON Schema、正则表达式、固定模板 输出，确保格式 100% 合法

CASE：生成JSON格式

CASE：生成JSON格式

假设我们希望语言模型生成一个故事中角色的JSON对象, 这个角色需要有一个名字和一个年龄, 分别对应JSON的两个Key, “name”和 “age”字段。

这里使用Pydantic定义一个角色：

Pydantic 是一个Python 库，用于数据验证和设置管理。

在 Outlines 工具中，Pydantic 可以高效地生成符合JSON Schema 或 Pydantic 模型的JSON 数据。

name=<Name.paul: 'Paul'> age=<Age.thirty: 30>



```
from enum import Enum
from pydantic import BaseModel

class Name(str, Enum):
    john = "John"
    paul = "Paul"

class Age(int, Enum):
    twenty = 20
    thirty = 30

class Character(BaseModel):
    name: Name
    age: Age

from outlines import models, generate

model = models.transformers("/root/autodl-tmp/transformers/models/gpt2")
generator = generate.json(model, Character)

char = generator("Generate a young character named Paul.")
print(char)
```

Outlines工具

Step1: 将JSON转换为正则表达式

正则表达式是处理结构化输出的重要的部分，当向Outlines传入Pydantic定义的Model时, Outlines首先会将Model转换成Json格式:

```
json_schema = Character.model_json_schema()
print(json_schema)
```

```
{'$defs': {'Age': {'enum': [20, 30], 'title': 'Age', 'type': 'integer'},
  'Name': {'enum': ['John', 'Paul'], 'title': 'Name', 'type': 'string'}},
 'properties': {'name': {'$ref': '#/$defs/Name'},
  'age': {'$ref': '#/$defs/Age'}},
 'required': ['name', 'age'],
 'title': 'Character',
 'type': 'object'}
```

```
import outlines.fsm as fsm
```

```
import json
```

```
# 将JSON schema转化为正则表达式
```

```
regex_str =
```

```
fsm.json_schema.build_regex_from_schema(json.dumps(json_schema))
```

```
print(regex_str)
```

```
\{[ ]?"name"[ ]?:[ ]?("John"|"Paul")[ ]?,[ ]?"age"[ ]?:[ ]?(20|30)[ ]?\}
```

我们可以利用这个正则表达式来控制LLM的输出结构化。理论上不是所有的JSON格式都能用正则表达式表示，但大多数情况下，用正则表达式近似表示就够用了。

Outlines工具

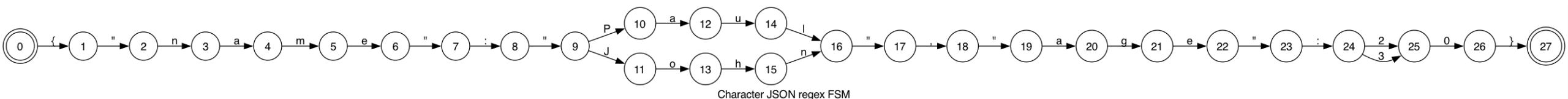
Step2: 正则表达式表达为有限状态机

Outlines结构化生成速度快，是因为正则表达式和有限状态机之间的等价性。

1) 使用interegular第三方库将JSON正则表达式转换为有限状态机。

2) 根据有限状态机进行Json结构的生成了，具体过程：

- 从状态0开始。
- 随机生成允许的转换字符之一。
- 按照相应的转换到达新状态。
- 重复以上过程，直到达到有限状态机的最终状态之一（这里只有一种最终状态，即状态27）。



使用interegular输出的有限状态机（可视化图）

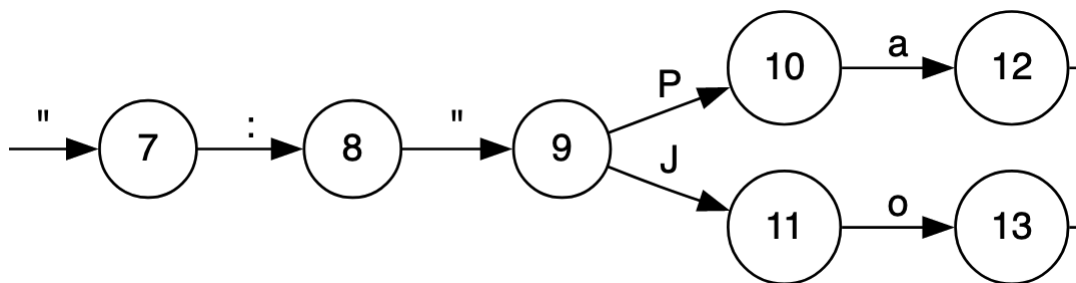
从0状态开始，0状态只能有一个允许转换的状态就是{，一直到9状态，都是固定的转移，到了第9个状态，才有的选，可以选P到达10状态，也可以选J到达11状态... 最后到达最终状态。

Outlines工具

通过合并（Coalescence）加速结构化的生成：

观察上述有限状态机（FSM）的部分，可以看到：

大多数状态只有一个可转移的状态。那么抽样就显得没有必要。



把只有一个转移状态的节点进行合并，我们就可以跳过一些采样过程了（i.e. 大模型的调用次数），合并之后，上面的有限状态机就变成了下面这样：



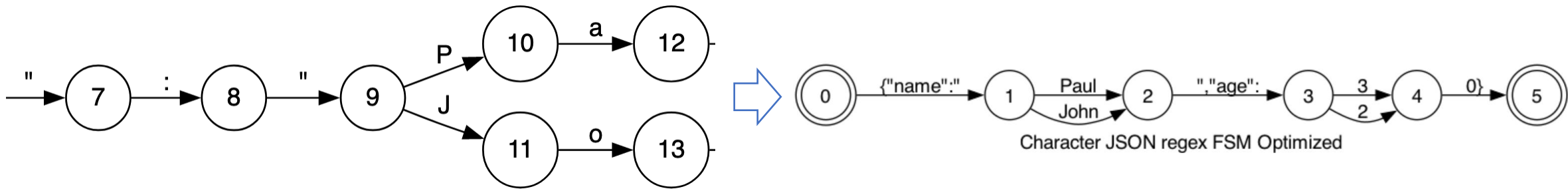
Outlines工具

假如Prompt是 "generate a character", 以前不需要结构化 有了上面的有限状态机, 就变成了下面的采样过程:

输出的时候, 正常的调用过程是: 第一轮: 输入: generate a character, 大模型输出&采样: y 出&采样: P

第二轮: 输入: generate a character, y 大模型输出&采样: 第二轮: 输入: generate a character, {"name", "Paul", "age": 大模型输出&采样: 3

第n轮: 输入: generate a character, yes i can generate a 然后就不用再调用大模型了, 直接得到generate a character for yo , 大模型输出&采样: u character, {"name", "Paul", "age"30"}



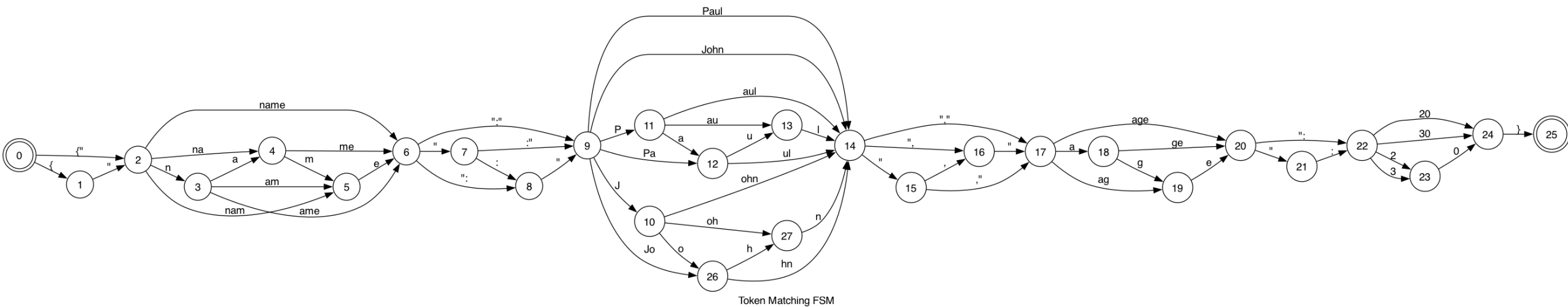
这里只调用2次大模型, 但是, LLMs不使用单个字母, 而是使用Token。所以还需要需要把字母变成token。

Outlines工具

LLM Works with tokens

Outlines可以将基于字符的有限状态机转换成基于Token的有限状态机。

看起来虽然很复杂，但是和上面单个字符的生成过程是一样的，只不过是把单个字符组合为token输入进大模型。



CASE: outlines使用

Outlines 通过 API 调用（generate.text）

当 outlines 通过 models.openai() 连接远端 API 时，只能使用 generate.text() 进行纯文本生成，无法使用 generate.json() 等结构化功能。

```
import os
import tiktoken
import outlines
import outlines.models as models

# outlines 内部使用 tiktoken 获取 tokenizer，需要为 qwen 模型注册兼容的编码器
tiktoken.model.MODEL_TO_ENCODING["qwen-max"] = "cl100k_base"

model_name = "qwen-max"
api_key = os.getenv("DASHSCOPE_API_KEY")
base_url = "https://dashscope.aliyuncs.com/compatible-mode/v1"
model = models.openai(model_name, api_key=api_key, base_url=base_url)
answer = outlines.generate.text(model)("你好，请介绍下自己")
print(answer)
```

你好！我是Qwen，由阿里云研发的超大规模语言模型。我被设计用来帮助人们生成各种类型的文本，如文章、故事、诗歌、故事等，并且可以完成多种语言任务，比如回答问题、提供建议、玩游戏等。我的目标是成为一个既有用又有趣的助手。如果你有任何问题或需要帮助，请尽管告诉我！

outlines 使用 tiktoken 获取 tokenizer，而 tiktoken 默认只识别 OpenAI 模型名。调用 qwen 模型时需要手动注册编码器映射：

```
tiktoken.model.MODEL_TO_ENCODING["qwen-max"] = "cl100k_base"
```

2-outlines使用1.py

CASE: outlines使用2

2-outlines使用2.py 在以上基础上用 @outlines.prompt 注入多条示例，并用 outlines.generate.text(model)(prompt) 调用；若把同一套 prompt 交给 OpenAI SDK 的 chat.completions.create，语义上等价，差别主要在是否采用 Outlines 的模板写法与调用链

```
examples = [  
    {"question": "37593 * 67 等于多少?", "code": "37593 * 67"},  
    {  
        "question": "小红的鸭子每天下16个蛋。她每天早餐吃3个，每天用4个  
做蛋糕给朋友。她把剩下的鸭蛋以每个2元的价格在市场上卖掉。她每天在  
市场上能赚多少钱?",  
        "code": "(16-3-4)*2",  
    },  
    {  
        "question": "做一件长袍需要2匹蓝色布料和一半数量的白色布料。总共  
需要多少匹布料?",  
        "code": "2 + 2/2",  
    },  
]
```

```
]  
question = "小明正在下载一个200GB的文件。他的下载速度是2GB/分钟，  
但下载到40%时下载失败了。然后小明不得不从头开始重新下载。他总共  
花了多少分钟下载完文件?"  
@outlines.prompt  
def answer_with_code_prompt(question, examples):  
    """"  
    {% for example in examples %}  
    QUESTION: {{example.question}}  
    CODE: {{example.code}}  
    {% endfor %}  
    QUESTION: {{question}}  
    CODE: """"
```

3-outlines使用2.py

CASE: outlines使用2

```
def extract_code(text):
    """从模型返回的文本中提取可执行的代码表达式"""
    import re
    # 尝试提取CODE:后面的代码
    match = re.search(r'CODE:\s*(.+)', text)
    if match:
        code_line = match.group(1).strip()
        # 去掉行尾注释
        code_line = re.split(r'\s*#', code_line)[0].strip()
        return code_line
    # 如果没有CODE:前缀，取第一行非空内容
    for line in text.strip().split('\n'):
        line = line.strip()
        if line and not line.startswith('#'):
            return re.split(r'\s*#', line)[0].strip()
    return text.strip()
```

```
prompt = answer_with_code_prompt(question, examples)

# 使用DashScope API(兼容OpenAI协议)
model_name = "qwen-max"
api_key = os.getenv("DASHSCOPE_API_KEY")
base_url = "https://dashscope.aliyuncs.com/compatible-mode/v1"

model = models.openai(model_name, api_key=api_key, base_url=base_url)

answer = outlines.generate.text(model)(prompt)
print('answer=', answer)
code = extract_code(answer)
print('code=', code)
result = eval(code)
print(f"下载文件总共花费 {result:.0f} 分钟")
```

CASE: outlines使用2

answer= 根据题目描述，小明下载一个200GB的文件，当下载进度达到40%时失败，之后不得不再次从头开始下载直至完成。我们先计算两次下载过程中所花费的时间总和。

- 文件大小为200GB。
- 下载速度为2GB/分钟。
- 第一次下载时，下载了文件的40%，即 $(200\text{GB} \times 40\% = 80\text{GB})$ 。
- 由于下载速度是2GB/分钟，因此第一次下载80GB所需时间为 $(80\text{GB} \div 2\text{GB/分钟} = 40)$ 分钟。
- 第二次重新下载整个200GB文件所需时间为 $(200\text{GB} \div 2\text{GB/分钟} = 100)$ 分钟。

因此，小明总共花费的时间为第一次下载时间加上第二次下载时间，即 $(40\text{分钟} + 100\text{分钟} = 140)$ 分钟。

所以，最终答案是小明总共花费了140分钟来下载完文件。

CODE: `40 + 100` # 将两次下载所耗时间相加得到总时间

code= `40 + 100`

下载文件总共花费 140 分钟

CASE: vllm内置outlines结构化

vLLM 在服务端内置了结构化输出引擎 (xgrammar / outlines)

不需要安装任何库，通过调用 vLLM API 时传入约束参数，即可在 token 采样层面强制输出符合指定格式。

流程：

客户端 (OpenAI SDK)，传入 response_format 或 structured_outputs 参数

=> vLLM 服务端

Pydantic 模型 -> JSON Schema -> 正则表达式 -> FSM

每一步 token 采样时，FSM 过滤不合法的 token

=> 输出 100% 符合约束的结构化内容

CASE: vllm内置outlines结构化

Thinking: Qwen3 系列默认启用思考模式，输出会以 <think>...</think> 包裹推理过程，导致结构化解析失败。

如何关闭 Qwen3 的思考模式？

方法1：请求级别

```
extra_body={"chat_template_kwargs": {"enable_thinking": False}}
```

方法2：服务启动级别

```
--default-chat-template-kwarg '{"enable_thinking": false}'
```

方法3：Prompt 级别

在 user 消息末尾追加 /no_think

CASE: vllm内置outlines结构化

客户端连接

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="dummy"
)

MODEL_NAME = "/root/autodl-tmp/models/qwen/Qwen3-0.6B"
EXTRA_BODY = {"chat_template_kwargs": {"enable_thinking": False}}
```

CASE: vllm内置outlines结构化

四种结构化约束方式

1. JSON Schema 约束（response_format）

通过 Pydantic 定义数据模型，自动生成 JSON Schema，传入 response_format 参数。vLLM 在服务端用 FSM 约束 token 采样 => 100% 保证输出为合法 JSON。

```
from enum import Enum
from pydantic import BaseModel
```

```
class Name(str, Enum):
```

```
    john = "John"
```

```
    paul = "Paul"
```

```
    lisa = "Lisa"
```

```
class Character(BaseModel):
```

```
    name: Name
```

```
    age: int
```

```
    occupation: str
```

```
response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {"role": "user", "content": "Generate a character named
Paul who is 30 years old and works as an engineer."}
    ],
    max_tokens=256,
```

```
    response_format={
```

```
        "type": "json_schema",
```

```
        "json_schema": {
```

```
            "name": "character",
```

```
            "schema": Character.model_json_schema()
```

```
        },
    },
    extra_body=EXTRA_BODY,
)
character =
Character.model_validate_json(response.choices[
0].message.content)
```

实际输出：

```
{
  "name": "Paul",
  "age": 30,
  "occupation": "Engineer"
}
```

4-vllm内置outlines结构化.py

CASE: vllm内置outlines结构化

2 正则表达式约束（structured_outputs.regex）

强制模型输出匹配指定正则表达式的内容，适用于 IP 地址、邮箱、手机号等格式化数据。

```
ip_regex = r"((25[0-5]|2[0-4]\d|[01]?[d\d?])\.){3}(25[0-5]|2[0-4]\d|[01]?[d\d?])"

response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {"role": "user", "content": "What is the IP address of Google DNS server?"}
    ],
    max_tokens=64,
    extra_body={
        **EXTRA_BODY,
        "structured_outputs": {"regex": ip_regex},
    },
```

实际输出：

1.1.0.66

正则约束只保证格式合法（符合 IP 地址的数值范围和点分格式），不保证语义正确。

这里模型没有输出预期的 8.8.8.8，但输出的 1.1.0.66 确实是一个合法的 IP 地址格式。

CASE: vllm内置outlines结构化

3 多项选择约束（structured_outputs.choice）

模型只能从给定的选项选择一个输出，适用于分类任务。

```
response = client.chat.completions.create(  
    model=MODEL_NAME,  
    messages=[  
        {"role": "user", "content": "这段评论的情感倾向是什么? 评论:  
这个产品质量很好, 非常满意!"}  
    ],  
    max_tokens=16,  
    extra_body={  
        **EXTRA_BODY,  
        "structured_outputs": {"choice": ["positive", "negative",  
"neutral"]},  
    },  
)
```

实际输出:

positive

模型从 ["positive", "negative", "neutral"] 三个选项中准确选择了 positive，与评论的积极语义一致。

CASE: vllm内置outlines结构化

4 复杂嵌套 JSON (response_format)

支持嵌套的 Pydantic 模型，适用于实际业务场景如订单提取。

```
class OrderItem(BaseModel):
```

```
    product_name: str
```

```
    quantity: int
```

```
    unit_price: float
```

```
class OrderInfo(BaseModel):
```

```
    customer_name: str
```

```
    items: list[OrderItem]
```

```
    total_price: float
```

```
response = client.chat.completions.create(
```

```
    model=MODEL_NAME,
```

```
    messages=[
```

```
        {"role": "user", "content": "Extract order info: Customer Zhang
```

```
Wei ordered 2 lanterns at $999.99 each and 3 mice at $29.99 each."}]
```

```
],
```

```
max_tokens=512,
```

```
response_format={
```

```
    "type": "json_schema",
```

```
    "json_schema": {
```

```
        "name": "order-info",
```

```
        "schema": OrderInfo.model_json_schema(),
```

```
    },
```

```
},
```

```
extra_body=EXTRA_BODY,
```

```
)
```

```
order =
```

```
OrderInfo.model_validate_json(response.choices[0].message.content)
```

4-vllm内置outlines结构化.py

CASE: vllm内置outlines结构化

实际输出:

```
{  
  "customer_name": "Zhang Wei",  
  "items": [  
    {"product_name": "Laptop", "quantity": 2, "unit_price":  
999.99},  
    {"product_name": "Mouse", "quantity": 3, "unit_price":  
29.99}  
  ],  
  "total_price": 2000.98  
}
```

解析后可以直接通过属性访问嵌套数据:

客户: Zhang Wei

商品: Laptop, 数量: 2, 单价: 999.99

商品: Mouse, 数量: 3, 单价: 29.99

总价: 2000.98

total_price 的正确值应为 $2 \times 999.99 + 3 \times 29.99 = 2089.95$ ，但模型输出了 2000.98。这再次说明结构化约束保证的是输出格式（字段名、类型、嵌套结构），而非数值计算的正确性。

=> 对于需要精确计算的场景，建议在拿到结构化结果后，由程序端重新计算。

结构化输出 (OpenAI兼容协议)

结构化输出

结构化输出:

DashScope(通义千问)提供了兼容OpenAI协议的API, 支持结构化输出。可使用qwen-turbo、qwen-max等模型。

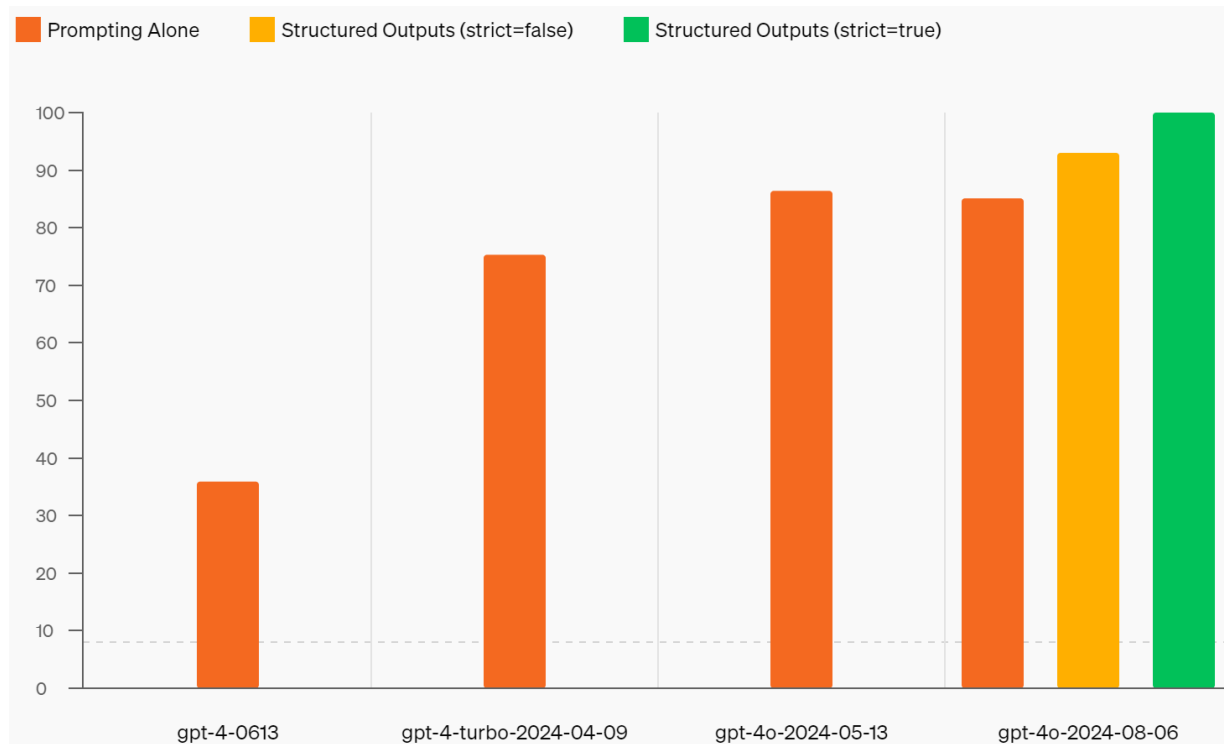
- Function calling模式

在函数定义中设置strict:true, 可以结构化输出。

- response_format参数

可以通过json_schema提供JSON Schema。当模型不是调用工具, 而是以结构化方式响应用户时, 很有用。需要提供响应格式并设置为strict:true。

说明: DashScope的OpenAI兼容接口支持Function Calling和response_format两种结构化输出方式, 使用方式与OpenAI SDK一致。



在复杂JSON的评估中, 设置 strict=true的结构化输出得分完美 100%

结构化输出

POST https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

```
{
  "model": "qwen-max",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful assistant. The current date is August 6, 2024.
You help users query for the data they are looking for by calling the query
function."
    },
    {
      "role": "user",
      "content": "look up all my orders in may of last year that were fulfilled but
not delivered on time"
    }
  ],
```

<https://openai.com/index/introducing-structured-outputs-in-the-api/>

```
"tools": [
  {
    "type": "function",
    "function": {
      "name": "query",
      "description": "Execute a query.",
      "strict": true,
      "parameters": {
        "type": "object",
        "properties": {
          "table_name": {
            "type": "string",
            "enum": ["orders"]
          },
          "columns": {
            "type": "array",
            "items": {
```

.....

结构化输出

结构化输出的安全性：

- 结构化输出将遵循现有的安全政策，允许模型拒绝不安全的请求。
- API 响应中新增了一个拒绝字符串值，能够以编程方式检测模型是否生成了拒绝，而不是输出匹配模式。
- 当响应中不包含拒绝且模型的响应没有被提前中断（如 `finish_reason` 所示）时，模型的响应将可靠地产生与提供的模式匹配的有效 JSON。

```
{
  "id": "chatcmpl-9nYAG9LPNonX8DAyrkwYfemr3C8HC",
  "object": "chat.completion",
  "created": 1721596428,
  "model": "gpt-4o-2024-08-06",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "refusal": "I'm sorry, I cannot assist with that request."
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 81,
    "completion_tokens": 11,
    "total_tokens": 92
  },
  "system_fingerprint": "fp_3407719c7f"
}
```

结构化输出

- 基于Python SDK的使用(DashScope/Qwen)

原生支持结构化输出。提供工具或作为响应格式的模式就像提供Pydantic对象一样简单。

SDK 会自动处理将数据类型转换为支持的 JSON 模式，将 JSON 响应反序列化为类型化的数据结构，并在出现拒绝时进行解析。

```
from enum import Enum
from typing import Union
from pydantic import BaseModel
```

```
import openai
from openai import OpenAI
```

```
# 定义数据库表的枚举类型
```

```
class Table(str, Enum):
    orders = "orders" # 订单表
    customers = "customers" # 客户表
    products = "products" # 产品表
```

```
# 定义数据库列的枚举类型
```

```
class Column(str, Enum):
    id = "id" # 唯一标识符
    status = "status" # 订单状态
    expected_delivery_date = "expected_delivery_date" # 预期交货日期
    delivered_at = "delivered_at" # 实际交货日期
    shipped_at = "shipped_at" # 发货日期
    ordered_at = "ordered_at" # 订单日期
    canceled_at = "canceled_at" # 取消日期
```

结构化输出

定义操作符的枚举类型

```
class Operator(str, Enum):  
    eq = "=" # 等于  
    gt = ">" # 大于  
    lt = "<" # 小于  
    le = "<=" # 小于等于  
    ge = ">=" # 大于等于  
    ne = "!=" # 不等于
```

定义排序方式的枚举类型

```
class OrderBy(str, Enum):  
    asc = "asc" # 升序  
    desc = "desc" # 降序
```

定义动态值的数据模型

```
class DynamicValue(BaseModel):  
    column_name: str # 列名
```

定义条件的数据模型

```
class Condition(BaseModel):  
    column: str # 列  
    operator: Operator # 操作符  
    value: Union[str, int, DynamicValue
```

定义查询的数据模型

```
class Query(BaseModel):  
    table_name: Table # 表名  
    columns: list[Column] # 查询的列  
    conditions: list[Condition] # 查询的条件  
    order_by: OrderBy # 排序方式  
# 创建DashScope客户端(兼容OpenAI协议)  
client = OpenAI(  
    api_key=os.getenv("DASHSCOPE_API_KEY"),  
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",  
)  
# 通过Function Calling进行结构化输出  
completion = client.beta.chat.completions.parse(  
    model="qwen-max",  
    messages=[  
        {"role": "system",  
         "content": "你是一个有用的助手。当前日期是2026年3月22日。你帮助用户通过调用查询函数来查找他们需要的数据。"},  
        {"role": "user",  
         "content": "查找去年5月所有已完成但未按时交付的订单"},  
    ],  
    tools=[  
        openai.pydantic_function_tool(Query), # 使用Pydantic模型作为工具  
    ],  
)  
# 打印解析后的查询参数  
print(completion.choices[0].message.tool_calls[0].function.parsed_arguments)
```


结构化输出

```
table_name=<Table.orders: 'orders'> columns=[<Column.id: 'id'>, <Column.status: 课件打印/code/2-structured-function-calling.py
```

```
table_name=<Table.orders: 'orders'> columns=[<Column.id: 'id'>, <Column.status: 课件打印/code/2-structured-function-calling.py
```

```
table_name=<Table.orders: 'orders'> columns=[<Column.id: 'id'>, <Column.status: table_name=<Table.orders: 'orders'>
```

```
columns=[<Column.id: 'id'>, <Column.status: table_name=<Table.orders: 'orders'> columns=[<Column.id: 'id'>, <Column.status:
```

```
'status'>, <Column.expected_delivery_date: 'expected_delivery_date'>, <Columtable_name=<Table.orders: 'orders'>
```

```
columns=[<Column.id: 'id'>, <Column.status: 'status'>, <Column.expected_delivery_date: 'expected_delivery_date'>,
```

```
<Ctable_name=<Table.orders: 'orders'> columns=[<Column.id: 'id'>, <Column.status: 'status'>, <Column.expected_delivery_date:
```

```
'expected_delivery_date'>, <Columlivered_at: 'delivered_at'>] conditions=[Condition(column='status', operator=<livered_at:
```

```
'delivered_at'>] conditions=[Condition(column='status', lived_at: 'delivered_at'>] conditions=[Condition(column='status',
```

```
operator=<Operator.eq: '='>, value='completed'), Condition(column='expected_delivery_date', operator=<Operator.lt: '<'>,
```

```
value='delivered_at'), Condition(column='ordered_at', operator=<Operator.ge: '>='>, value='livered_at: 'delivered_at'>]
```

```
conditions=[Condition(column='status', operator=<Operator.eq: '='>, value='completed'), Condition(column='expected_delivery_date',
```

```
operator=<Operator.lt: '<'>, value='delivered_at'), Condition(column='ordered_at', operator=<Operator.ge: '>='>, value='2023-05-01'),
```

```
Condition(column='ordered_at', operator=<Operator.le: '<='>, value='2023-05-31'))] order_by=<OrderBy.asc: 'asc'>
```

结构化输出

```
import os
import json
from pydantic import BaseModel
from openai import OpenAI

class Step(BaseModel):
    explanation: str
    output: str

class MathResponse(BaseModel):
    steps: list[Step]
    final_answer: str

# 创建DashScope客户端(兼容OpenAI协议)
client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
)

# 在system prompt中描述JSON格式, 配合response_format=json_object使用
schema_desc = json.dumps(MathResponse.model_json_schema(),
    ensure_ascii=False, indent=2)
```

```
# 通过response_format进行结构化输出
completion = client.chat.completions.create(
    model="qwen-max",
    messages=[
        {
            "role": "system",
            "content": f"你是一个数学辅导老师。请严格按照以下JSON Schema格式返回解题过程:\n{schema_desc}",
        },
        {"role": "user", "content": "解方程  $8x + 31 = 2$ "},
    ],
    response_format={"type": "json_object"},
)

# 手动解析JSON为Pydantic模型
message = completion.choices[0].message
result = MathResponse.model_validate_json(message.content)
print(result.steps)
print(result.final_answer)
```

设置response_format完成结构化 6-输出结构结构化.py

结构化输出

```
[Step(explanation='首先，我们需要将方程中的常数项移至等式的一边。为此，我们从两边同时减去31。',  
output='8x + 31 - 31 = 2 - 31'), Step(explanation='简化上述表达式后，得到新的方程形式。', output='8x = -29'),  
Step(explanation='接下来，为了解出x的值，我们需要将方程两边都除以8。', output='8x / 8 = -29 / 8'),  
Step(explanation='完成最后一步计算，得出x的具体数值。', output='x = -29/8 或 x ≈ -3.625')]
```

$x = -29/8$ 或者 $x \approx -3.625$

CASE：食谱助手

```
from pydantic import BaseModel
from openai import OpenAI
```

食材

```
class Ingredient(BaseModel):
```

```
    name: str
```

```
    amount: str
```

```
    kcal: int
```

食谱

```
class Recipe(BaseModel):
```

```
    ingredients: list[Ingredient]
```

```
    instructions: str
```

```
# 创建DashScope客户端(兼容OpenAI协议)
```

```
client = OpenAI(
```

```
    api_key=os.getenv("DASHSCOPE_API_KEY"),
```

```
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
```

```
)
```

```
# 在system prompt中描述JSON格式
```

```
schema_desc = json.dumps(Recipe.model_json_schema(), ensure_ascii=False, indent=2)
```

```
# 通过response_format进行结构化输出
```

```
completion = client.chat.completions.create(
```

```
    model="qwen-turbo",
```

```
    messages=[
```

```
        {
```

```
            "role": "system",
```

```
            "content": f"你是一个食谱助手。请严格按照以下JSON Schema格式返回结
```

7-食谱助手.py

CASE：食谱助手

```
果:\n{schema_desc}",
    },
    {"role": "user", "content": "请写一个苹果派的食谱"},
],
response_format={"type": "json_object"},
)

apple_pie_recipe =
Recipe.model_validate_json(completion.choices[0].message.content)

print(apple_pie_recipe)
```

ingredients=[Ingredient(name='苹果', amount='6个', kcal=120), Ingredient(name='糖', amount='1杯', kcal=770), Ingredient(name='肉桂粉', amount='1茶匙', kcal=10), Ingredient(name='面粉', amount='1/4杯', kcal=120), Ingredient(name='黄油', amount='1/2杯', kcal=400), Ingredient(name='水', amount='2汤匙', kcal=0)] instructions='1. 将苹果去皮、去核并切片。2. 在一个碗中，将苹果、糖、肉桂粉和面粉混合。3. 在另一个碗中，将黄油和水混合，然后加入到苹果混合物中。4. 将混合物倒入派皮中，并在预热至180°C的烤箱中烘烤约45分钟，直到表面呈金黄色。5. 让其冷却后即可享用。'

打卡：结构化信息输出



对你的LLM 应用进行结构化信息输出，比如：

- Function Calling结构化
- 返回结果结构化
- 通过Pydantic 进行结构化约束
- 你也可以使用 vllm 进行结构化信息约束

观察输出的结果，是否信息结构更加稳定



Thank You
Using data to solve problems